Coral-Chords documentation and user guide

Linus Tibert

September 7, 2025

Contents

1 Used abbreviations

Although some of those might seem obvious, I'll give a complete overview of special abbreviations used in the document for those who are not familiar with any of them.

CCh	Coral-Chords
UG	Ultimate Guitar
BE	backend
FE	frontend
UI/GUI	(graphical) user-interface
URL	uniform resource locator (web-link)
API	application programming interface
ID/UID	(unique) identifier

2 Introduction

This document is supposed to be a useful guide for those who want to understand how the code of CCh works. It is especially designed to be an explanation of how each component of the program is working, however, some detailed parts (such as specific `functions()`) are not mentioned here, because they are easier to understand (if you know what the whole section of the code is for), when looking at the actual code. The code is widely commented and should be relatively easy to understand.

To make this document more accessible for everyone, it is divided into many small sections, each with a clear subject.

3 Project structure

The project is divided into two major parts: the backend (referred to as BA), and the main program (called the frontend or FA).

The FE is the part of the program which interacts directly. Thus, it mainly creates and updates the GUI using the Iced GUI library. Some UI-builders are split into separate files to increase the maintainability of the code.

The BA is where the magic happens. Its purpose is to do anything not directly involving the user. Thus, it fulfils tasks involving networking, data extraction or direct interaction with the (file) system.

3.1 Backend

Because the BA still has many tasks, it is subdivided into a bunch of submodules defined in the code. I'll explain the tasks of each of them in the following sections.

3.1.1 UG scraper module

This module is used to retrieve data from UG and process it into useable data.

This module was originally developed as a local library just for this project, but after it was finished, it got uploaded publicly as a separate crate to crates.io and GitHub.[1]

3.1.2 System module

As the name suggests, every direct interaction with the user's system is managed by this module. It is mainly used to read and write files.

3.1.3 Types module

All globally used types and constants are defined here.

This module mainly exists to keep the code clean. Having all custom types in one module makes it easier to manage them across all parts of the project.

[1] https://github.com/Lich-Corals/ug-scraper-rs/

4 Functionality details

4.1 Getting the data from UG

Ultimate Guitar does not have an API a publicly available API. I.e., there is no easy way for developers outside the company to use their site as a data source. Thus, the best way to get UG's data is to get their source HTML and run a couple of RegEx patterns on it to get raw data.

This is called "web scraping" and is exactly what CCh, or the `ug_scraper` library does. This way CCh gets a very limited kind of API to get machine processable data from UG. In the following sections I will elucidate the way CCh scrapes the data from raw HTML on UG.

4.1.1 Legal aspects of web scraping

First, a tiny disclaimer: Web scraping is not illegal, and in this case it is legal too.

The important thing is that CCh is not downloading/scraping any restricted or confidential data. All it really does is downloading the same contents your web browser is downloading as soon as you look at a tab or search result at UG.

Besides this, CCh does not contain any scraped (i.e., copyrighted) data; with publishing this project, I only supply people with an easy method of getting UG's tabs in a custom UI without re-publishing any of the copyrighted materials of UG.

To read more about this topic, you can visit my source of those facts in the footnote.[2]

4.1.2 Scraping UG's data

Everything explained in this section is happening in the `ug_scraper` crate. Thus, if you want a more technical view of it, I recommend taking a look at the crate at crates.io or GitHub.

What the crate is doing to get data from UG can essentially be described with three steps:

1. Download data from UG
2. Extract valuable information
3. Output data in a custom format

In the first step, a simple function is triggered which returns the raw HTML of the selected UG page. (This page is either a tab or a search results page.) To make sure the downloaded data is valid for data extraction, the program simply checks for a few strings within the HTML; if certain ones are present and certain ones are not present, the page is valid.

The data extraction is done with a few RegEx patterns. Those are basically collections of special characters telling the computer to look for a specific pattern in a given piece of data. Within those patterns, certain groups are defined, telling the RegEx engine which parts of the pattern are the wanted data.

[2] https://blog.apify.com/is-web-scraping-legal/

As soon as the data is extracted, it is serialized and saved as a file in YAML format.

When the saved song is now requested by the user (by playing it on Spotify), the YAML is deserialized and drawn to the user interface with the configured look.

4.2 Interaction with Spotify

CCh interacts with Spotify using the MPRIS, a standard D-Bus interface for communication between Linux applications and media players.

Using MPRIS, CCh obtains the UID of the currently playing song, which is chosen by Spotify. The IDs of songs aren't human-readable, but they are unique, which makes them useable for reliably identifying a song. To connect the song ID and tab, CCh saves the serialized tab in a file named with the song's UID. E.g.: `4PTG3Z6ehGkBFwjybzWkR8.yml`.

5 Usage guide

This section will give you instructions on how to use and configure CCh.

5.1 Basic usage

To be able to use CCh, you need to run it on a system with MPRIS support and Spotify installed.

All you need to do after launching the application is to toggle the "Play" switch in the head bar. Now the application will automatically check if there is a tab is already available for this song.

5.1.1 Searching for songs

If there is no song available to show, the view will automatically switch to "Search" and launch a search for the detected song title and artist. To change the search query, you can "Clear" the search bar, type your own query and hit "Go!" to launch a new search.

If you only want a specific type of tab, you can select a filter on the right or "Reset" the filter to get all results again.

When you've found a tab you want to download, you can hit "Download" next to the title and switch to the "Tab" view. This button may be inactive, which means, the tab format is not supported for downloading. Note that the tab downloaded will be saved under the ID of the currently playing song on Spotify, regardless of which title the song has.

If you're not happy with the downloaded tab, you can download any other one. This will overwrite the currently stored file. To see the new tab, press the "Reload" button in the "Tab" view.

5.1.2 Editing tabs

To edit a tab, hit the "Edit..." button in the "Tab" view while the song to edit is playing. This will launch your default text editor for you to edit the contents of the file.

The contents of the file have to be in YAML format, if this in not the case, loading the song will return "Bad YAML data" errors.

Within the `lines:` part of the file will be repeating `- line_type:, text_data:` pairs. This structure has to be maintained, while editing, adding or removing lines.

Supported `line_type`s are the following: `Lyric`, `Chord` and `SectionTitle`. The following line of `text_data` will be displayed as the defined type.

The `metadata` and `basic_data` sections are used to store general information about the song. Some metadata entries may have `null` as a value, which means they don't store any data.

When editing any of those entries, the original data types have to be maintained. This means a value containing a number without quotes shouldn't be changed to anything other than that. All data stored in the metadata section has the text type, which means the values should either be numbers in quotes or text without quotes. Every metadata entry may also be set to `null` to remove it.

To apply any changes, save the file and "Reload" in the "Tab" view.

5.2 Keyboard shortcuts

Coral-Chords provides a few keyboard shortcuts for easier usage of the application:

Key-code	Action
Media player	
Space	Play/pause
2x Space	Rewind to 0 and pause
3x Space	Rewind to 0 and play
Application	
Enter	Toggle play mode
1	Tab page
2	Search page
3	Settings page
F1	Edit tab
F7	Clear notifications
F8	Reload tab
F9	Welcome page

5.3 Configuration

This subsection will tell you everything you need to know about each configuration option offered by CCh.
All settings are automatically saved in a configuration file on change. The most common location for this file is `/.config/coral-chords/config.yml`. All settings can be manually changed in the file or by changing them using the GUI.

5.3.1 Application

You can select one of 22 predefined themes for the application. The default theme is “Dark”.

If you don’t want to get notified about new available versions of Coral-Chords, you can disable the “Notify about updates” setting.

The application logs your played songs in a local file by default. If you don’t want this log file, you can disable the “Log played songs locally” feature. Note that CCh never sends any of this data to anyone; this feature is merely existing to enable interested users to evaluate their musical history.

CCh is able to remind you to change your instrument’s strings regularly. To enable this setting, tick the corresponding box. The interval slider below is set to three months by default but you can change it to your needs.

5.3.2 Search

If you choose to “only show downloadable search results”, all search results which can not be downloaded will be excluded from the list. This is turned off by default.

The “Auto-clean” feature will automatically remove things like “ - Remastered 2019” from search queries when entering them automatically. This is on by default.

The “Search depth” describes how many results will be listed when searching. The number is not meant literally; it is a multiplier, meaning there will be about 20 additional results for each number more than 1. The default value is 2.
Note that a higher search depth may lead to much longer searching times when a lot of results are available.

5.3.3 Tabs

“Remove lines before first chords” will remove the first lines of a displayed tab which only contain text data. With this feature enabled, longer tabs will fit on the screen and information like “Tabbed by Jane Doe” may be hidden. This setting is enabled by default.

“Remove empty lines” will compress displayed tabs dramatically at the cost of readability by removing any lines without any text data. This is normally disabled.

The “Text size” setting will change the size of text seen in the “Tab” view. The default size is 15.

The “Header colour”, “Metadata colour” and “Chord colour” settings change the colours of certain elements of a displayed tab. Hex-colour-codes with and without alpha channel are accepted. To restore the default colours, empty the input fields.